

Received August 23, 2018, accepted October 16, 2018, date of publication October 31, 2018, date of current version December 3, 2018.

Digital Object Identifier 10.1109/ACCESS.2018.2878894

Distributed Reinforcement Learning in Emergency Response Simulation

CESAR LOPEZ¹, (Student Member, IEEE), **JOSÉ R. MARTÍ**¹, (Fellow, IEEE),
AND SARBJIT SARKARIA

Department of Electrical and Computer Engineering, The University of British Columbia, Vancouver, BC V6T 1Z4, Canada

Corresponding author: Cesar Lopez (ceslopez@ieee.org)

ABSTRACT This paper presents the implementation of a coordinated decision-making agent for emergency response scenarios. The agent's implementation uses reinforcement learning (RL). RL is a machine learning technique that enables an agent to learn from experimenting. The agent's learning is based on rewards, and feedback signals proportional to how good its actions are. The simulation platform used was infrastructure interdependencies simulator, in which, we have tested suitability of the approach in previous studies. In this paper, we have added new features to our previous solution, for enabling faster convergence and distributed processing. These additions include an enhanced reward scheme and a scheduler for orchestrating the distributed training. We include two test cases. The first case is a compact model with four critical infrastructures. In this model, the agent's training required only 10% of the attempts needed in our previous version. Improvements in convergence come from adding a shaping reward scheme. We trained the agent across 24 simultaneous configurations of our model. The training process elapsed 4 min. The extended case included more infrastructures and a higher level of detail. The dimensionality of the problem grew by a factor of 4000, but the training converged in less episodes. We tested the extended model over 96 parallel instances (potential scenarios) with completion in 2.87 min. The results show a fast and stable convergence. This agent can help during multiple stages of emergency response including real-time situations.

INDEX TERMS Distributed reinforcement learning agent, interdependencies simulation, decision making in emergency response, complex systems, disaster response.

I. INTRODUCTION

The frequency of natural disasters is increasing. Three main contributors to this trend are: growing population and infrastructures, vast population growth on coastal areas and urbanizing risk-prone areas [1], [2]. As population grows, more people can be affected by disasters. People seem to have more awareness of disasters happening and their negative impact on society. Moreover, news travels faster than before and spreads significantly. An aspect that facilitates the spreading of disaster news is the expanding availability of the Internet and social networks [2]. The evolution of Emergency Management, as a discipline, and its systematic contribution to increasing the effectiveness of first responders has been an important factor in minimizing the impact of disastrous events. Emergency Management is defined in [3] as: *"The discipline and profession of applying science, technology, planning and management to deal with extreme events that can injure or kill large numbers of people, do extensive damage to property, and disrupt community life"*.

Emergency Management, as a process, is seen as a continuum as it cycles through four interdependent stages [4]:

- **Prevention/Mitigation:** is an attempt to keep hazards from becoming disasters, or to reduce their consequences. Mitigation efforts pursue long-term actions to manage (reduce/remove) the risk [5].
- **Preparedness:** getting ready to respond to disasters and to manage their negative consequences by using contingencies taken prior to an event, i.e. emergency plans, training, etc. [4]. This first half of the emergency response continuum exists without an incident.
- **Response:** all emergency response activities start from the moment an incident strikes and end once the situation following impact has been stabilized [6].
- **Recovery:** comprises all those measures taken to bring back communities to pre-emergency conditions, or even a better status, after an emergency or disaster has been detected and response has ended [7].

Traditionally, emergency management in Canada has focused on preparedness and response [4] but as observed in Figure 1, the emergency management cycle is an unending process or interrelated steps [8].

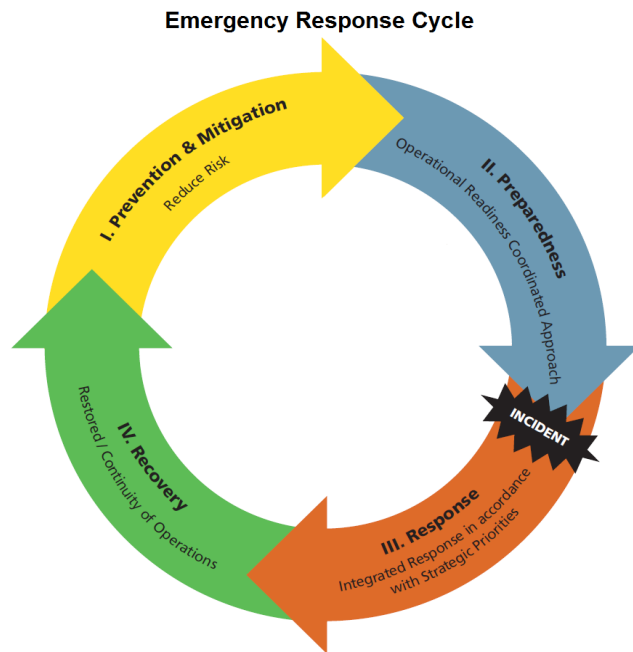


FIGURE 1. Emergency management cycle. Defined in the Canadian emergency management planning guide 2010-2011 [8].

Making decisions during a disaster is one of the most critical tasks responders perform, given that the timeframe is limited and, lives and property are at stake. Governments keep enhancing their response plans and strategies, but one thing is common to all of them, considering Critical Infrastructures (CIs) as foundational. The U.S. Department of Homeland Security, in [9], defines CIs as the backbone of their economy, security and health. CIs are highly interdependent, and a coordinated response is necessary to avoid poor decisions amplifying the original impact of hazardous situations.

The simulation platform used in this study allows the representation and analysis of Critical Infrastructures' Interdependencies. Critical Infrastructures (CIs) are essential to the well-being of people and their failures can cascade with disastrous consequences.

The implementation described in this work shows fitness for use at any of the stages of the emergency management cycle. During *prevention/mitigation*, tasks related to identifying the sensitivity of CIs and evaluating their resilience can be carried out in order to devise ways to increase robustness of those CIs having the highest level of criticality. In relation to *preparedness*, simulation of past events enables managers and planners to evaluate the outcome of decisions made, while responding to those events and, to test other actions that could potentially lead to better results. These lessons learned are a good insight towards improving action plans and procedures for first responders. For these two previous stages the time

to produce results does not follow tight constraints. During *response*, the time window to identify a suitable action(s) is very limited and it wouldn't allow more than a few minutes. Hence, a real-time simulation environment capable of determining best course of action is ideal. *Recovery* uses knowledge gathered during previous stages and evaluates a positive cascading effect aiming at prioritizing restoration of infrastructures in a way that minimizes the total restoration time. The proposed simulation framework has been tested with scenarios that reflect all the activities mentioned before. i2Sim, the Infrastructure Interdependencies' Simulator, is the tool to be used for this study, details about this platform will be provided later.

The remaining part of this paper is organized as follows: Section II introduces critical infrastructures and their interdependencies, along with methodologies for simulating them. Section III describes i2Sim and explains its capabilities for simulating interdependent and complex systems. Section IV provides an overview of reinforcement learning (RL) a machine learning technique used in this work for training decision making agents. Section V presents a proof of concept test case for decision making via a distributed reinforcement learning approach. In Section VI, a model with large dimensionality and three independent agents is analyzed through simulation, this model expands the proof of concept. Section VII collects conclusions and final remarks.

II. CRITICAL INFRASTRUCTURES AND INTERDEPENDENCIES

Critical infrastructure (CI) is defined as: “*processes, systems, facilities, technologies, networks, assets and services essential to the health, safety, security or economic well-being of Canadians and the effective functioning of government*” [10] by Public Safety Canada in the National Strategy for Critical Infrastructure. This definition considers similar components as the ones provided by countries like US, Australia, New Zealand and UK. This can be found in a state of the art in critical infrastructure protection and resilience collected through a project supported by Defense Research and Development Canada (DRDC) [11]. The US Department of Homeland Security specifies 16 critical infrastructure sectors in [12]. Similarly, the Government of Canada defines 10 Critical Infrastructures (CIs): energy and utilities, information and communication technology, finance, health, food, water, transportation, safety, government and manufacturing. The Canadian critical infrastructure sectors are shown in Figure 2.

As found in [11], all the listed nations (Canada, US, Australia, New Zealand and UK) included energy, water, transport, ICT, health, food supply, banking/finance, government services, safety/emergency services. The results of this report also list modeling & simulation, risk assessment, decision making/support, policy & directives, and monitoring & warning as mitigation strategies. Additionally, the literature collected in [11] shows an increasing interest in disasters, particularly in natural disasters.

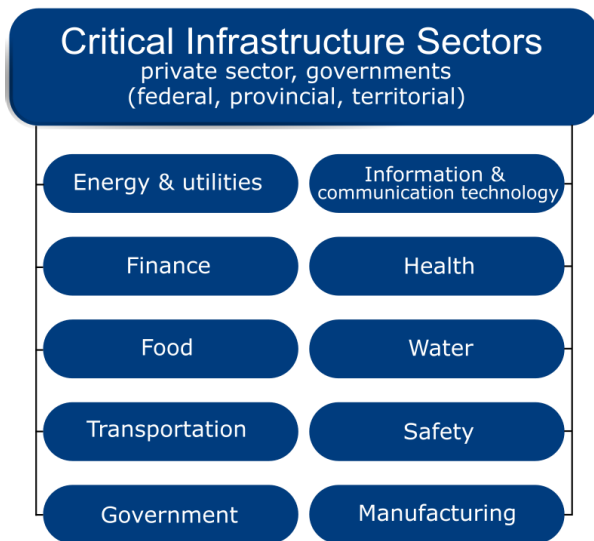


FIGURE 2. Critical infrastructure sectors in Canada. Source: Public Safety Canada.

Critical Infrastructures (CIs), while clearly defined as individual entities, don't operate in isolation. Instead, they are intrinsically connected at levels that sometimes are not observable at first glance. Those relationships can help identifying vulnerabilities in the systems as well as tracing cascading effects after incidents impact the system.

Setola and Theocharidou [13] provide a thorough review on dependencies among infrastructures. Their work characterizes these relationships based on their order; where order is determined by the proximity of the other entities affected by the incident as it propagates. The authors explain how dependencies might have loops and reciprocal influence. These dependencies are called interdependencies. Moreover, the authors describe 5 categories for interdependency analysis: physical, cyber, geographical, logical, and social.

Individual infrastructures are complex on their own and having them interconnected elevates the complexity of their analysis. As single infrastructures are systems and, they are inherently interdependent in conjunction, the study turns into a system of systems modeling with the aim of unveiling those hidden bidirectional relationships (interdependencies) [14].

We can see that when dealing with CIs, decisions are effective if they are not taken in isolation. Hence, for an effective coordination, an understanding of how the systems' components interact is required to minimize the consequences of hazardous events affecting complex scenarios [15].

A. MODELING OF CRITICAL INFRASTRUCTURES' INTERDEPENDENCIES

Modeling and Simulation are two important tools because they enable the planners and managing (practitioners) teams to evaluate potential risks and vulnerabilities.

Ouyang [16] provides a survey on Modeling and Simulation of Interdependent CIs, this overview provides a

walkthrough of the inherent relationship of both terms in the context of Disaster Response. The author highlights the importance of interdependencies in disruptive scenarios, as some of the interrelations are hidden in normal conditions but made obvious under critical conditions.

De Nicola *et al.* [17] highlight the importance of simulation in crisis management and technical disasters. The authors introduce CEML (Crisis and Emergency Modeling Language), a behavioral modeling framework. CEML focuses on reactive behavioral descriptions where a set of actions are performed as something happens, hence the two key ideas are to receive "Abstract Service Stimuli" and to respond with "Abstract Service Response". While the idea is to connect the behavioral language CEML with agent-based simulators, it does not seem as robust as machine learning techniques, applied to CI simulation environments, like Reinforcement Learning (RL).

Foglietta *et al.* [18] introduce the concept of holistic and reductionist simulation models. Their work highlights the importance of mixing both approaches, however, while using a holistic approach all interactions among CIs are disregarded and proper representations of interconnected infrastructure might not be possible.

Tofani *et al.* [19] introduce phenomenological simulation. Phenomenological simulation offers more abstract representations to be added to models as different components (modules) interact in a system of systems fashion. Among phenomenological analysis the authors list the most common approaches used in practice. These methodologies include topological analysis, where the emphasis goes on how elements connect more than their dynamics. A second method is input and output systems, having components, like infrastructures, represented by a block with a specific input/output mapping. Likewise, the authors include agent base modeling as an option, where different components interact based on a given behavior, and the agents can provide optimizations based on expert knowledge or acquired knowledge of the environment.

III. THE i2Sim MODELING FRAMEWORK

i2Sim, the Infrastructure Interdependencies Simulator, is a simulation framework developed at the Complex Systems Integration Centre at the University of British Columbia.

The Infrastructures' Interdependencies Simulator (i2Sim) is a tool that allows the representation of interconnected critical infrastructures and modeling of their interdependencies. Furthermore, through the simulation process some hidden interdependencies can be revealed [20]. To understand how the simulating tool works, we provide an overview of the most significant features. We describe the layered architecture, the component ontology, and two key components for this study: the production cell, and the distributor.

A. LAYERED ARCHITECTURE

i2Sim has been developed as a stack of layers. Members of this stack are interchangeable by means of internal routines or

external applications via APIs. Having this level of abstraction promotes independence of tasks, where each layer deals only with its specific role while treating the remaining layers as black boxes. Points of communication between layers are limited, in order to guarantee more control over message exchange. The list of layers includes:

1) LOW LEVEL LAYER

This layer refers to specialized tools outside i2Sim. The low level layer manages ways of interacting with specific domain simulators e.g., electrical simulators like PSCAD or PowerWorld, among others.

2) PHYSICAL LAYER

The main layer in i2Sim. It provides the user with a series of blocks for representing the infrastructures and their topological connections. With the representation of the physical entities, this layer performs infrastructure interdependency evaluation and clock synchronization. This layer follows an ontology that uses a unified set of components capable of representing a diversity of scenarios.

3) DAMAGE ASSESSMENT LAYER

This layer interacts with external sensor data and performs damage assessment on infrastructures. It connects to the physical layer, and pushes updates containing the status of the physical simulation components. The damage assessment layer can also act as a bridge between the physical layer and specialized damage assessment tools like BCSims [21].

4) DECISION LAYER

This layer has a bidirectional channel of communication with the physical layer. It allows it to capture the instantaneous state of the physical components and to choose the best action according to the established goals. As the main task of decision makers, in disrupted scenarios, is to allocate resources, the output of this layer is a set of ratios transferred to a component called the distributor. The decision layer can use different types of algorithms among which Artificial Intelligence (AI) agents are a preferred choice. Previous work evaluated multiple approaches including Monte Carlo, reinforcement learning (RL), ordinal optimization and genetic algorithms. However, RL seems to adapt better to the simulator's implementation due to the high level of discretization natural to the representation of most components. RL, in its basic form, uses a matrix-based look-up table (LUT) to store the acquired knowledge; for that reason dimensionality is a big concern. The Distributed RL agent proposed addresses that problem while delivering a faster training process.

B. ONTOLOGY

i2Sim's ontology was created to provide the right level of generalization different scenarios and infrastructures require. This ontology has a simplified set of universal components used to represent multiple real objects [22], [23]. The components are grouped by affinity as follows:

1) TOKENS

The tokens are the units that flow through the system. A token is a quantity that is transferred from one component to another. Hence, tokens are the inputs and outputs of every component and always travel through a *channel*.

2) CELLS

Cells are the production units. The cells have the power of creating, transforming and storing tokens. The output tokens may be of the same nature as the inputs, or a transformation of combined heterogeneous resources.

3) CHANNELS

Channels are a means to transport tokens from element to element. Channels can configure a time delay. The time delay is the amount of time tokens remain inside the channel. This way, complex interconnections like roads, pipes, etc., can be expressed as the time it takes for tokens to travel from their origin to their destination. As a convention, every link between two elements must be done by using a channel.

4) CONTROL ELEMENTS

The control elements are those allowing the user to establish decision points, tuning the simulation parameters, and controlling the simulation engine. Decision points relate to resource allocation and distribution. These elements provide a communication link with upper layers.

5) VISUALIZATION ELEMENTS

Visualization elements allow the user to probe specific system outputs and to present the results on two-dimension axes against time. Results may be presented as a matrix of individual or overlapped plots.

C. THE PRODUCTION CELL (PC)

The Production Cell is a component, in the i2Sim library, used to represent the CIs in models. It is implemented as a block and the input/output mapping is commonly done via a Human Readable Table (HRT). The HRT is a table that can be easily generated by users and enables them to model the behavior of complex infrastructures which are highly nonlinear. HRTs, as defined in i2Sim, have a set of inputs and a single output. The inputs correspond to resources needed for the correct operability of the infrastructure represented but they could also be modifiers. Modifiers are special type of inputs that allow the inclusion of human factors e.g., medical personal fatigue.

The PC's operability modes are defined by two variables: Physical Mode (PM) and Resource Mode (RM). The physical mode represents the physical integrity of the facilities, i.e., their capability to operate. The RM is an instantaneous specification of the level of input resources available. The input (resource) with the lowest value is called the *limiting factor* as it determines the output. Figure 3 illustrates the

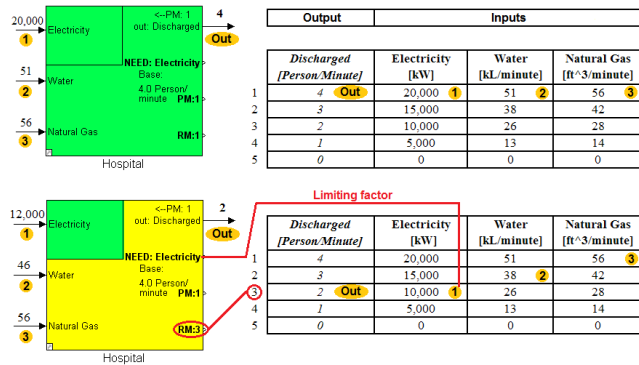


FIGURE 3. Operation of the production cell based on a Human Readable Table (HRT). It can be seen how a change in the inputs lowers operability of the infrastructure from 100% to 50%.

concept of the limiting factor and the changes made to the infrastructure's output when the inputs are changed.

The production cell displays the PM as a colored rectangle in the upper-left corner, and as a value in its third output port. The resource mode is reported as the solid color filling the cell itself, and its value is passed to the fourth output port. All these details are visible in Figure 3, details about color mapping and combined PM/RM operability are treated later on.

The PC belongs to the physical layer and it offers connectivity with upper layers. While input resources come from the same layer, the physical mode is provided by the damage assessment layer. Likewise, the current PM and RM values are sent to the decision layer, as state variables, for decision agents to determine the status of the environment.

D. THE DISTRIBUTOR

The distributor is an element that splits an input signal into two or more output signals. The process is done by assigning a ratio to each one of the outputs. The sum of all ratios is 1 (100%) i.e., the value of the input. The distributor's behavior can be fixed or controlled by an agent from the decision layer. With a fixed behavior, the distributor has a permanent set of ratios throughout the simulation. When an agent controls a distributor, a message from the decision layer brings the instantaneous configuration of the distributor, the settings remain until new ones are provided.

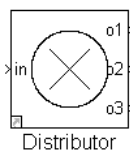


FIGURE 4. The i2Sim Distributor. Splits the input among 2 or more outputs. With 3 outputs it works like this: $o1=X1(in)$, $o2=X2(in)$, and $o3=X3(in)$. $x1+X2+X3=1$.

IV. AN OVERVIEW OF REINFORCEMENT LEARNING (RL)

This overview aims at collecting foundational concepts about RL necessary for understanding the formulation of the test

cases used for the study. To start talking about RL the first step is to discuss Markov Decision Processes (MDPs).

A. MARKOV DECISION PROCESSES (MDPS)

A Markov Decision Process (MDP) is a tuple $\langle S, A, T, R \rangle$ where S is a finite set of states, A a finite set of actions, T refers to a transition function, and R is a reward function.

1) STATES

A state is a mapping of the status of the environment. It is unique and determined by significant features from the environment. Some MDPs might contain illegal states, in this work, all states are considered legal. States are a finite set $\{s_1, \dots, s_N\}$ with N elements.

2) ACTIONS

An action is what the agent can do at any point in time, and, in the current state. Actions are a finite set including K options $A = \{a_1, \dots, a_K\}$. For this project, all actions are possible (legal) in all states.

3) THE TRANSITION FUNCTION

Determines the probability on ending in state s' when action a is applied during the current state s . The transition function is, formally, defined as $T: S \times A \times S \rightarrow [0, 1]$. In other words, it is about knowing in what state you end up if you choose a certain action. In *Model Free* implementations, like the ones included in this paper, there is no initial knowledge about state transition for representing it mathematically [24]. Hence, all transitions are led by exploration.

4) THE REWARD FUNCTION

The reward function is a numerical feedback signal derived from the agent's choices, as such, it implicitly specifies the goal of the learning process. For model free implementations, the reward function is defined as $R: S \times A \times S \rightarrow \mathbb{R}$, or $R(s, a, s')$. In this case, the reward signal is calculated based on the transition from the initial state s to the resulting state s' after applying action a .

In Reinforcement Learning, the Markov property makes the system depend on previous states and actions only through the current state of the system [25]. Hence, the environment's response at time $t + 1$ depends only on the state and action at step t . Given that all the information needed to make a decision is provided by the current state, without the need of history terms, the Markov property has been also called *Independence of Path*. An environment is said to have one-step dynamics with independence of path, this enables the agent to predict the next state and reward given only the current state and action [26], as shown in (1).

$$Pr \{s_{t+1} = s', r_{t+1} = r \mid s_t, a_t\} \quad (1)$$

The optimality of an MDP is based on collecting the best rewards along the timeline, hence, maximizing the return. Considering the future is important and it can be done in

different ways. However, for this study we used *discounted-finite horizon*. Following this approach, future rewards are considered but discounted depending on how far ahead they are. Thus, rewards obtained later are discounted more than the ones received earlier. The estimated return is discounted infinite horizon is given by:

$$E \left[\sum_{t=0}^{\infty} \gamma^t r_t \right] \quad (2)$$

In (2) γ is the discount factor, it takes values from 0 to 1, as $0 \leq \gamma < 1$. E is the expected return, a summation of all rewards collected over time. When the agent does not consider future rewards, γ is 0 and the agent is said to be *myopic* [24].

B. POLICIES

A *Policy* is a prescription to the decision agent, telling it what action to choose for any possible future state [25]. Therefore, the policy is the mapping between states and actions, and it is defined as a function that takes state $s \in S$ as input and outputs an action $a \in A$. There must be an output value associated to each state. The policy is, commonly, denoted as π . This work is based on deterministic policies, where function π maps the set S into the set A , as $\pi : S \rightarrow A$.

C. STATE-ACTION VALUE FUNCTION (BELLMAN'S EQUATION)

Value functions are used for determining how beneficial a specific action is in the current state [27]. A value function is used to calculate the value of taking action a in state s and while following policy π . It is expressed as $Q^\pi(s, a)$, for this reason it can also be called a *Q Function* [26]. Equation (3) shows the Q function as a function of the pair state-action. Each value is estimated as the expected return following policy π , E_π , over every current state (s_t) and action (a_t).

$$Q^\pi(s, a) = E_\pi \left(\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | s_t = s, a_t = a \right) \quad (3)$$

The optimal value function, denoted by $Q^*(s, a)$, provides maximum values for all the states and it can be obtained by solving the next *Bellman equation* [28]:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a') \quad (4)$$

In (4), we can see the optimal policy expressed as the current reward plus the discounted future rewards following a converging policy. The solution of (4) gives the optimal policy π^* :

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a) \quad (5)$$

Equation (5) show the optimal policy as a function of the state, where the output for every state is the action providing the maximum return.

A Q function enables the agent to learn the optimal policy by exploring the environment rather than having to

know the full dynamics of the environment. Exploration is key for acquiring knowledge, but as the learning progresses the accumulated knowledge is to be used. This opens a big discussion in Reinforcement Learning, the *exploration-exploitation dilemma*. Exploring refers to trying random actions, while exploiting is using the already known “good” actions. A good balance between exploration and exploitation is needed for achieving optimal solutions.

In the absence of exploration, the agent might get stuck in suboptimal solutions (local maxima) achieved in the developing policy. Exploration opens the opportunity of discovering new paths leading to a global maximum.

A good rule to balance exploration and exploitation is the ϵ -greedy approach, where ϵ is the exploration factor, e.g. $\epsilon = 0.1$, means that 10% of the movements are exploratory while 90% are greedy, i.e. choosing the highest value. In this project we reduce ϵ as the training process progresses.

Reinforcement Learning can be solved with three different algorithms: Dynamic Programming (DP), Monte Carlo (MC) or Temporal Difference (TD) methods. DP bases new knowledge on previously learned estimates, but it requires a full model of the environment's dynamics. MC does not need a full model, but the updates happen until the end of every episode. TD combines their strengths, updates are done online, and a full model of the environment is not needed. As emergency response simulations include a diversity of infrastructures, a complete model of the environment is nearly impossible. Likewise, making decisions under pressing conditions, with lives at risk, requires fast answers. These two factors were key when choosing the RL approach to be used. We have used a TD method for this project.

D. TEMPORAL DIFFERENCE METHODS

Temporal Difference (TD) methods are implemented as *on-line, fully incremental* methods, i.e., updates take place at $t+1$ (at every timestep) during the same episode. Every update is based on what happened during the previous point in time (t). This is known as *Bootstrapping* [26]. The most popular TD methods are *Sarsa-On Policy* and *Q-Learning*. We chose Sarsa-On policy due to its safe guaranteed convergence.

E. SARSA ON-POLICY

As a TD method, Sarsa updates as *sample backups*. Sample backups involve looking ahead for a sample successor state-action pair, then use that future value along with the reward for computing a *backed-up value* for the original state [26]. Sarsa is a TD(0) because it considers only the next ($t+1$) state-action pair. Thus, the update uses the quintuple $\langle S_t, a_t, r_{t+1}, S_{t+1}, a_{t+1} \rangle$, which gives the name to the method. The recursive Q value calculation for Sarsa on-policy looks like this:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \quad (6)$$

Equation (6) introduces a new variable, the learning rate α . The learning rate determines how much of the arriving knowledge you add to what you already know. The learning rate takes values from 0 to 1, where 0 means no learning and 1 means keeps only new lessons. Our previous experimental results have shown that $\alpha = 0.5$ and $\gamma = 0.7$ are a good combination, thus, they will be kept constant in all our cases.

Sarsa is an On-Policy TD estimation method because the future sampling is done following the current policy. A demonstration of Sarsa(0) convergence is available in [29].

F. SHAPING REWARDS

To achieve a faster optimal policy finding, additional rewards can be given to the agent as hints. These rewards are called *Shaping Rewards* and require a lot of trial and error. Poorly chosen rewards will lead the agent to learn poor solutions [30]. Poor choices for additional rewards could impair the agent. Shaping functions are more common to multi-criteria Reinforcement Learning, but equally applicable to single goaled implementations.

A *Shaping Reward Function* behaves in the same way as the reward function, defined before, and it is added to the original return [30]. The new reward function will be:

$$R' = R + F = R(s, a, s') + F(s, a, s') \quad (7)$$

V. PROOF OF CONCEPT TEST CASE

This case uses a compact model. Several assumptions were taken, aiming at minimizing the implementation complexity while providing a foundational case for a proof of concept. With i2Sim the user can specify any level of detail to scenarios or subsystems. Certain subsystems can include a more detailed construction, proportional to their impact on the study's objective.

For this test model, similar infrastructures have been grouped e.g. multiple electrical substations can be treated as a single electrical supply. This level of resolution is enough to represent the basic dynamics of the selected scenario.

This model includes four interconnected infrastructures: electrical substation, water station, stadium (venue), and the hospital. Electricity is supplied to all (other) infrastructures, water is supplied to stadium, hospital, and residential areas.

This scenario assumes a disruptive event affecting the stadium and producing injuries to a total of 3000 people. These people require medical attention at a hospital. The simulation starts when the disruptive event has already ended. This problem focuses on resource allocation, during the response stage of the emergency management continuum [4]. Attendees are evacuated and TRIAGED [31] on site, those in need of clinical treatment are transported to the emergency facilities.

In some of our previous projects we focused on earthquakes [20], [22], [32]. For this study, we give no specification of the type of disaster. We feel that the training under multiple configurations, in parallel, allows the agent to learn a wider policy applicable to a variety of incidents.

Most of the simulation span is devoted to finding the best resource allocation (electricity and water) for maximizing the output (treatment) at the hospital. A group of the state mappings suffers from scarce resource availability; hence, allocation of resources is more challenging. These instances constitute the main goal of the agent's training. The *best* states are the ones running without physical damage, and with total resource availability. These best states are used as a baseline, for verifying the convergence of the solution before extending it to a distributed implementation. The aggregate test model is presented in Figure 5.

A. STATE SPACE

As described in section IV, the states are determined by features that configure an instantaneous snapshot of the environment. The variables used for identifying the states are the status descriptors of the infrastructures. The availability of resources is discretized into 5 levels with a separation of 25%, these levels are called Resource Modes (RM).

Following the resource mapping, the physical integrity of each production cell (infrastructure) goes from 0% to 100% in 25% steps, this is called Physical Mode (PM). The physical mode contains the resource modes. The first physical mode (100%) contains 5 resource modes. As the physical damage increases the number of resource modes decreases. To represent the hierarchical relation between physical state and input resources, the PM and the RM were unified as a single operating index (OI). Each production cell has 15 possible configurations, as seen in Figure 6.

As the four PCs will follow the same setup (15 operational modes), a combinatorial of their operational indexes determines the size of the state space, 15^4 . Thus, the agent can map 50,625 states.

B. ACTION SPACE

After sensing the status of the environment, the agent reacts by choosing an action from a list of physically and topologically possible actions. Since this problem focuses on coordinated response, the agent's actions are simultaneous electrical and water distribution settings. A total of 5 actions have been configured at the electrical distributor and 3 at the water distributor. A combinatorial of both sets yields a total of 15 actions. All actions are valid for all the states. The two decision points have been highlighted in Figure 5.

This project uses a tabular state/action mapping, a look up table (LUT). The LUT is sized: $50,625 \times 15 = 759,375$. Each element in the LUT is indexed as $Q(s, a)$ and typed double (8 bytes). Therefore, the total memory space required for the LUT is $759,375 \times 8 = 6,075,000$ i.e., 5.79MB. This problem does not require a lot of memory, but it provides a suitable test case. When more details are added to the model, the dimensionality is likely to increase at an exponential rate. Hence, a distributed representation becomes a must if intending to use the platform for recreating realistic scenarios.

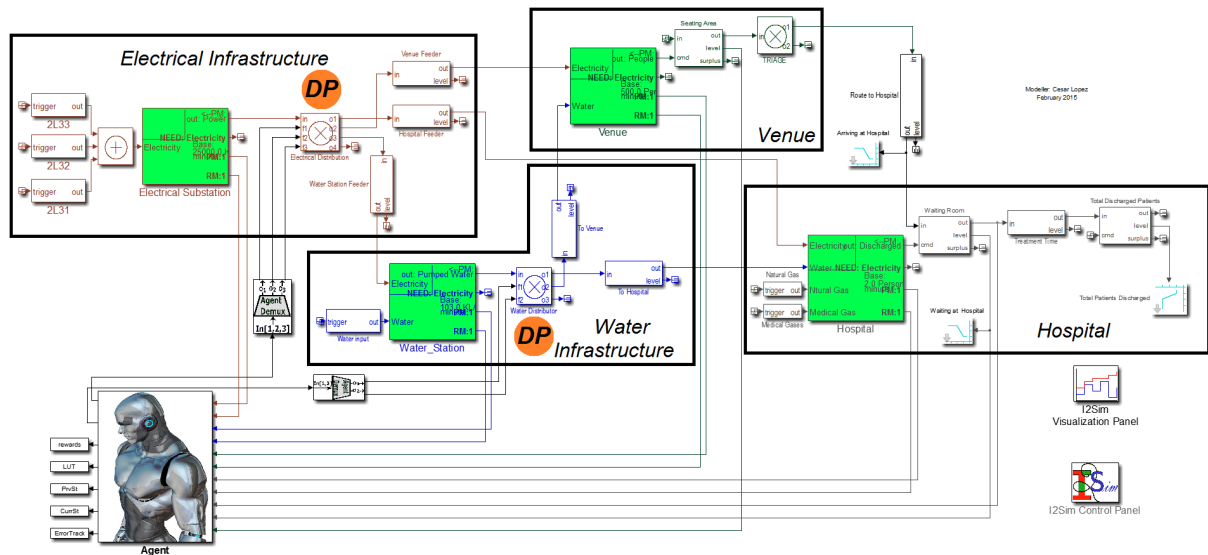


FIGURE 5. Aggregated test case. Four infrastructures are modeled, including their interconnection. A single agent acts as decision maker with two decision points (DP): electrical and water distribution. This model does not include a high level of detail and it is used as a proof of concept.

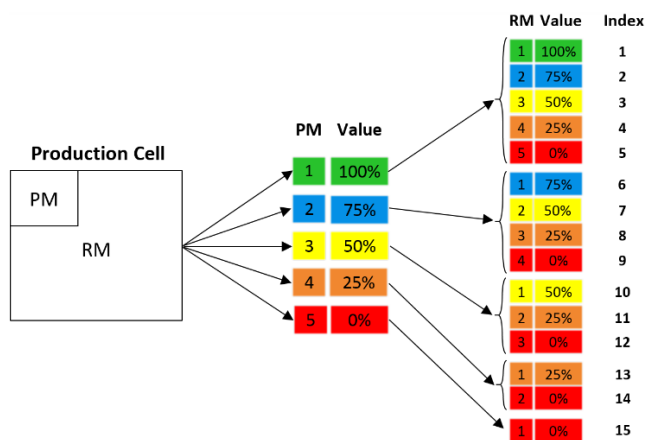


FIGURE 6. Production cell (PC) mapping to states based on the operating index (OI). This operating index combines physical modes with their resources modes as a single value from 1 to 15. The mapping between values and colours is also illustrated.

C. REWARDS

The main goal of the agent is to allocate the resources in a way that maximizes the number of people treated at the hospital. The instantaneous reward is calculated with a comparison between hospital's output at times t and $t - 1$. This rewards scheme was tested in past projects [33], [34]. Models with a similar configuration as the one described here, showed slow convergence.

A revised reward scheme was needed for speeding up convergence. We decided to add a shaping reward function. Some shaping rewards tested, related to transportation and egress tasks, but they did not help with the learning process.

As the simple reward function calculated the reward signal by merely subtracting the current hospital's output from its

previous output, that difference had a small value. The first step was to amplify that difference with a penalty factor 5, to make it significant. This, however, did not help with a zero difference. A zero difference means that the latest action produced no change to the hospital's output. We used the current hospital's output as the reward signal for this case. One last possibility was to have a zero difference and a zero output. Thus, a zero reward signal. We analyzed this option by asking: *if the hospital's waiting area is not empty, why no patients are treated?*. This last branch, of the possible reward options, sent a reward of -5 to the Qfunction. Our revised reward scheme is depicted in Figure 7.

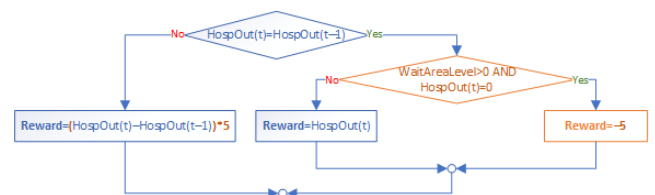


FIGURE 7. Improved reward scheme. Blue portions belong to a previously used reward scheme. Orange parts are the newly added improvements.

Our new reward scheme increases speed of convergence by a great factor, as discussed in the next item.

D. SEQUENTIAL SETUP AND RESULTS

The model spans over 1560 discrete steps corresponding to 1560 minutes of real time (26 hours). This is what we refer to as an episode or a full run. The aim of this test is to have the best sequential solution. A parallel solution is valid only when compared to its best sequential version. We will discuss four items at this point: look up table, convergence, policy, and results.

1) LOOK UP TABLE

As mentioned before, this implementation uses a tabular mapping of states and actions, a *Look Up Table* (LUT). The LUT is a matrix with actions as rows and states as columns.

Commonly, the LUT is initialized with a zero in all elements. In some of our previous tests we discovered a condition we have called *convenient convergence*. What we call convenient/coincidental convergence is finding the right action for a state as conveniently located. As an example, consider a LUT having zero as the initial Qvalue for all positions. The priority for selecting any action, in a particular state, is the same. As the agent looks for the highest Qvalue sequentially, action #1 would be selected. If action #1 happens to be the best one, for that state, it would be conveniently located. Thus, the solution would converge by a mere coincidence, leading to questionable results.

For proving strong convergence, we have used random values between -0.5 and 0.5 as initial Qvalues.

2) CONVERGENCE

Our previous work took 50 episodes, on average, to determine a policy [33]. With our new approach, 5 runs are enough to converge to the desired policy. The desired output is known for the baseline training scenario; thus, we can confirm the agent's convergence. We allowed the agent an extra training episode for increasing the robustness of the solution. The complete training includes 6 episodes.

A look at the initial episode shows the impact of the shaping rewards. This improved feedback helps the agent move towards the right solution (Figure 9, bottom) much faster than using the simple reward scheme. A comparison of the two reward schemes is illustrated in Figure 8.

3) POLICY

We let the agent explore new actions 5% of the times, at most, as a high level of exploitation is needed for SARSA to converge. As exploration decreases, episodically, we chose $\epsilon = \{5, 3.7, 2.5, 1, 0.5, 0.2\}$ as our exploratory scheme. Table 1 summarizes the details of the scheme.

TABLE 1. ϵ -greedy scheme.

Run#	ϵ	Greedy	#Exploratory actions	Explores every () steps
1	5%	95%	78	20
2	3.7%	96.3%	58	27
3	2.5%	97.5%	39	40
4	1%	99%	16	98
5	0.5%	99.5%	8	195
6	0.2%	99.8%	4	390

As seen in Table 1, every episode has a different (reduced) exploration rate. This rate is calculated in relation to the number of timesteps in an episode, 1560.

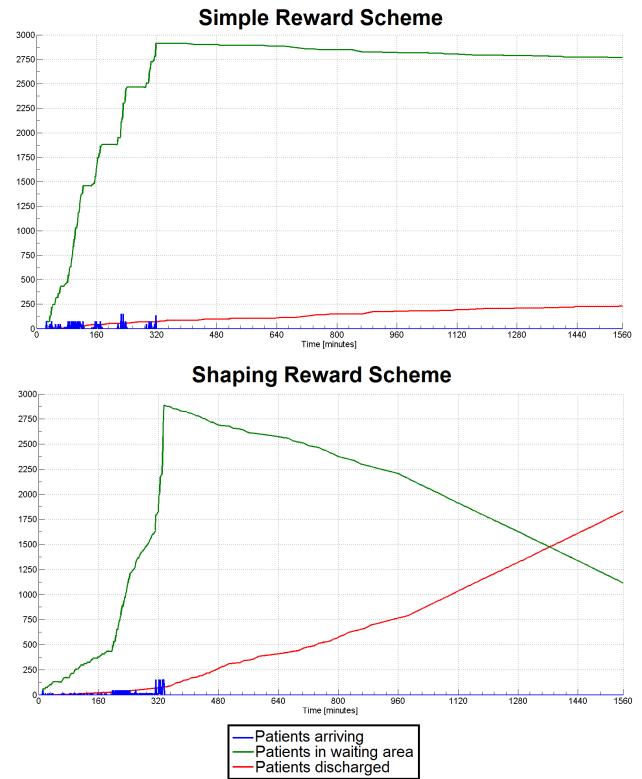


FIGURE 8. First training episode. Above the results of a simple reward scheme. Below, an improved reward scheme with shaping rewards.

To summarize the reinforcement learning setup and training, we provide the algorithm running inside the agent:

```

Initialize LUT ( $Q(s, a)$ ) with values between  $-0.5$  and  $0.5$ 
REPEAT (for each step of the simulation)
  IF timestep > 1 AND waiting area NOT empty THEN
    Calculate  $s_t$ 
    Compute  $r_t$  with reward function
    Choose  $a_t$  policy  $\in$  -greedy
     $Q(s_{t-1}, a_{t-1}) \leftarrow Q(s_{t-1}, a_{t-1}) + \alpha [r_t + \gamma Q(s_t, a_t) - Q(s_{t-1}, a_{t-1})]$ 
     $s_{t-1} = s_t$ 
     $a_{t-1} = a_t$ 
  ENDIF
UNTIL simulation is finished
  
```

The learning process starts from the second timestep as variables from $t - 1$ are used to generate feedback signals. Likewise, the learning is stopped when there are no more patients to treat (empty waiting area).

4) RESULTS

On average, an episode took 96 seconds to complete. The training was repeated over the baseline case and some of its variants 50 times, for guaranteeing consistent convergence. The agent learned the same policy across all tests. As specified before, the training set comprised 6 episodes. This testing was validated through the graphical user interface of the simulator.



FIGURE 9. The upper plot shows ripples and changes in slope. These variations suggest that the agent is trying different actions. Episode by episode the signal smooths down, as the agent converges to a policy.

The baseline model was configured to complete treatment of all injured people a few timesteps before completion. This was intentionally done for giving the agent just enough time to achieve the goal while a graphical validation was in place. Figure 9 shows a comparison of the results of the first and last episodes of the sequential version. With this sequential version calibrated, we proceed with the distributed implementation.

For the distributed implementation all graphical interfaces were disabled, and a new scheduler was programmed to manage the parallel training.

E. DISTRIBUTED IMPLEMENTATION

Finding decoupling points for the test case was the first step in generating a distributed solution. An understanding of the agent-environment interaction shows that the actions taken by the agent change the status of the simulation. But, not all state generating variables are linked to a decision point in the model, hence, those are suitable partitioning points. Those variables are CI variables that remain unaffected when agent allocates resources. The agent manages electrical and water supply distributions, by using a set of predefined actions, topologically correct.

The states are determined by a combination of physical and resource modes from all the CIs. Since no action taken by the

agent changes the physical integrity of any CI, the physical modes are state descriptors immune to the agent's acting. Likewise, the agent manages the split of electricity and water at the output of each corresponding facilities but does not control their inputs. Based upon these conclusions, we chose the decoupling variables to be the physical modes of all infrastructures and the resource modes of electrical and water infrastructures. We generated a partitioning table to be passed to the running instances as a parameter. This table was created by combining the decoupling state variables. We conducted a preliminary test over different configurations of the model and discovered that the agent cannot find a policy for states where electrical availability is below 50%. The corresponding partitions were removed from the table. It is important to highlight that a small test case, like this, allows that simplification. Larger cases with hundreds to thousands of combinations would be impossible to filter. The final table included 24 partitions of the LUT, enabling the same number of independent running instances of our model.

TABLE 2. Sample partition table (band).

Partition	OI1	OI2	OI3	OI4	State	# States
1	1	1	1	1	1	1125
	1	5	15	15	1125	
2	1	6	1	1	1126	900
	1	9	15	15	2025	
3	1	10	1	1	2026	675
	1	12	15	15	2700	
4	1	13	1	1	2701	450
	1	14	15	15	3150	

As observed in Table 2, each partition contains a specific number of states. The pattern showed in Table 2 repeats six times in our full table. We decided to call this repeating group of partitions a *band*. Thus, the partition table for this model has 6 bands. As the number of partitions matched the number of available computing nodes, a one to one mapping was possible. In larger cases, the association between instances and nodes could require a matching by bands instead of partitions. The listed OI values correspond to the operational indexes, previously, illustrated in Figure 6.

Looking for ways that allowed us to run the training in parallel, we considered different options. We evaluated big data frameworks like Spark and Hadoop, but we could not fit our agent due to the strict constraints in mapping partitions to processes performance-wise. A second option included an annual MATLAB parallel processing license, in the range of thousands of dollars. As the previous options were not feasible, we wrote our own scheduler for handling the distributed agent's training.

1) THE SCHEDULER

The scheduler is a set of routines to automate the training of the RL agent across multiple partitions of the state space, simultaneously. Our research lab is equipped with an IBM High Performance Computer (HPC) cluster with

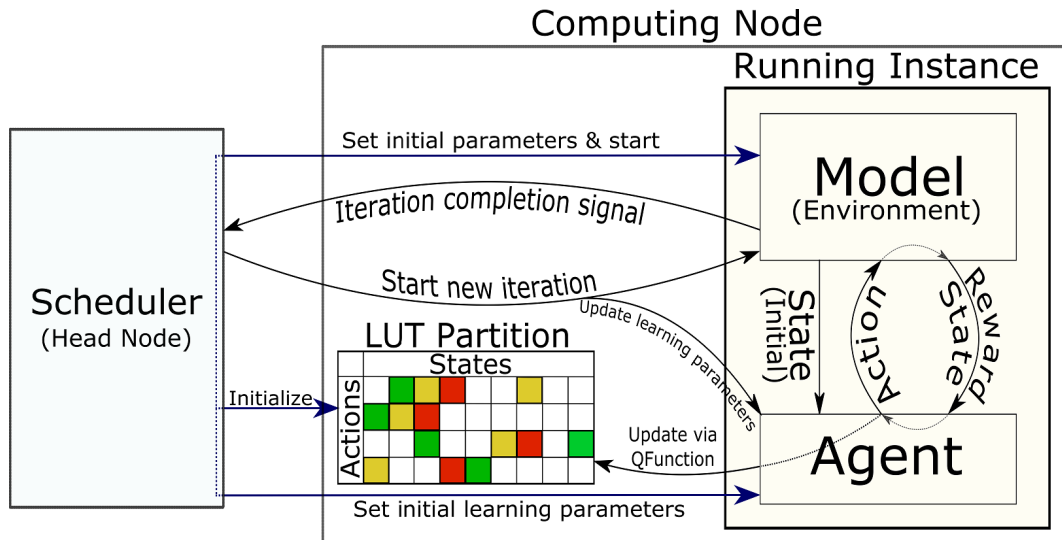


FIGURE 10. Scheduler architecture. A running instance per computing node works on a conventional RL optimization. Each instance has a local LUT (partition), updated every timestep, based on agent's experience. Every node has a managing routine, part of the scheduler, doing simulation in a loop and adjusting learning parameters. The external layer of the scheduler resides in the head node of the cluster. It starts the process with initial parameters and passes the control the local scheduler routines. When the control comes back from the distributed processes, the scheduler collects all the knowledge (local LUTs).

24 computing nodes. To minimize the communication overhead, we separated the scheduler implementation between the clustering software and the simulation platform. The clustering software used was xCAT, Extreme Cluster/Cloud Administration Toolkit, developed by IBM in 1999 and turned into open source in 2007 [35]. xCAT is a framework that allows management of HPC clusters. i2Sim was implemented as a MATLAB/Simulink toolbox using level 2 blocks. The scheduler performs the following tasks: setup model and learning parameters, looping execution of instances and gathering results from all nodes. The scheduler architecture is depicted in Figure 10.

1) *Setup of model and learning parameters*: a copy of the model is preloaded in each node along with the ϵ -Greedy scheme, partition matrix, and initial look up table. As we had a one to one mapping between partitions and computing nodes, the scheduler assigns each partition to the matching node, e.g., Node01 works the first partition. At that point xCAT passes the control to MATLAB by spawning all the threads via a parallel shell.

2) *Looping execution of instances*: the scheduler operates in MATLAB's scope, now, while xCAT awaits completion of the running threads. Each running instance is an independent sequential training. The scheduler iterates each instance 6 times, model parameters do not change as partitions remain unaltered. At the start of each iteration the scheduler modifies the learning parameters following the chosen ϵ -Greedy scheme, and each agent populates its LUT (partition).

3) *Gathering of results*: upon completion of the episodic iterative process the control is passed back to xCAT, the scheduler then collects all partitions.

After multiple tests in the cluster, the process was stable, and the results were retrieved correctly. The complete agent's

training (all instances) elapsed 4.21 minutes, at most. This is the highest time from all nodes. As expected, the variation was minimum, as all nodes had the same configuration and the simulation model was the same. The minimum running time was 3.26 minutes. The average training time for the agent was 4.06 minutes.

VI. EXTENDED TEST CASE

The extended test case uses a model with a more detailed representation. In this extended version we simulated an incident affecting two venues hosting simultaneous events. Both stadia located in the same geographical area in Downtown Vancouver, and at full capacity. The scenario is a complex interdependent system and comprises a set of subsystems as follows (see Figure 11):

1) *Electrical system*: the electrical supply is modeled with 4 substations and a bypass line. The substations are driven by an HRT that reflects the number of transformers in use. This way, each substation is discretized by nature. As an example, if the substation has three transformers, the possible levels of operation are 100%, 67%, 33%, or 0%.

2) *Water pumping station*: This subsystem considers the pumping station as a single (aggregate) cell with an input coming from the three watersheds supplying the Greater Vancouver Area. This infrastructure could be represented with a continuous function, but as Sarsa(0) uses a discrete set of states; a five level HRT: 100%, 75%, 50%, 25% and 0% was used instead. Water allocation focuses on supplying the hospitals, hence residential customers are out of scope.

3) *Venues (Egress)*: two stadia at full capacity going through a process of evacuation. Each venue is represented with two PCs using the same inputs. The first PC determines the rate of egress (time to leave the facilities). The second

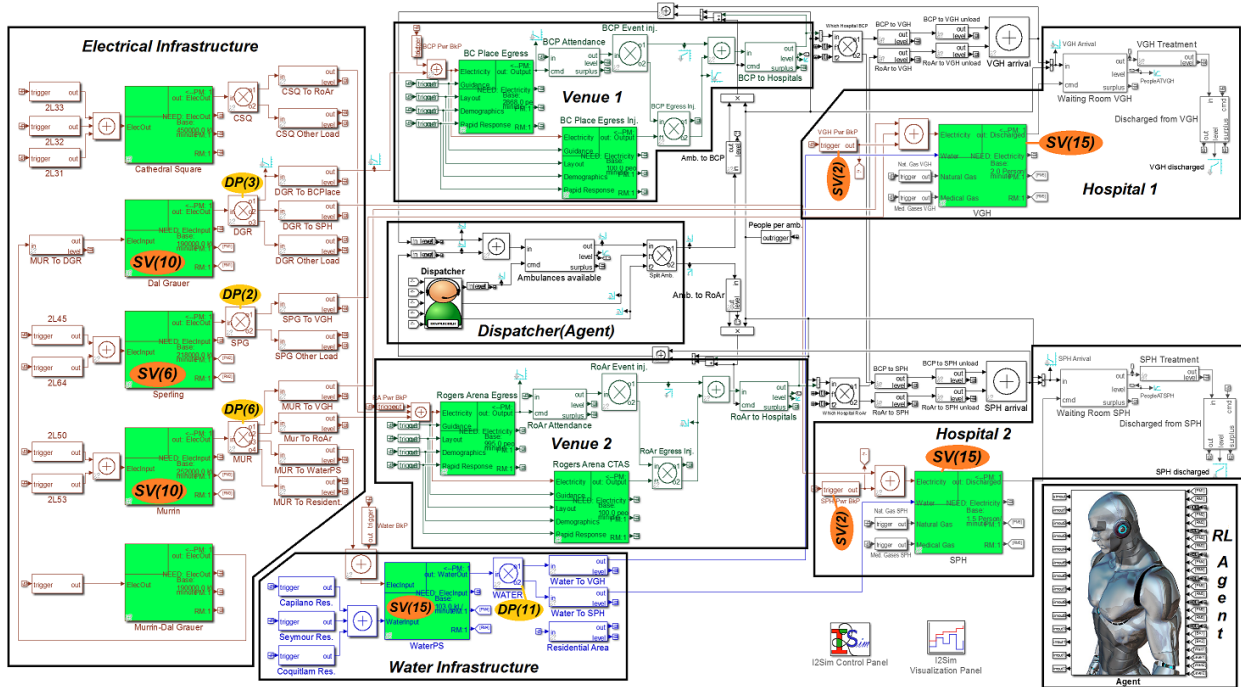


FIGURE 11. Extended test case. This model has more infrastructures than the proof of concept case, but not all of them are used for mapping states. Cells marked with SV have state space variables, the value in the parenthesis corresponds to the numbers Operational Indexes (OIs) available. Distributors managed by the RL agent are identified with DP and the number of possible combinations.

one calculates the amount of people injured due to the event. The inputs for a venue are resources and modifiers. Resources relate to physical supplies, i.e., electricity; while modifiers account for human factors. In this work, egress modifiers are: guidance, layout, demographics and rapid response. We have tested these modifiers in [32]. Egress happens even in the absence of resources. At egress, people are TRIAGED on-site. Those in need of medical attention will await transportation to emergency facilities.

4) *Transportation*: it is represented by channels joining ambulance stations, venues and hospitals. The traveling time has been determined using Google maps with the real location of venues and hospitals. We assume low traffic as the roads modeled, are designated disaster response routes.

5) *Hospitals*: are represented with PCs determining the rate of treatment based on the amount of available resources. Hospital HRTs were build with information collected from interviewing hospital managers and practitioners. The hospitals have different capacities, one of them is larger.

A. THE DECISION LAYER (AGENTS)

The decision layer has three independent, non-interfering agents. The agents are assigned to egress (dispatcher), transportation (On-site supervisor), and resource allocation (RL agent). The egress and transportation processes occur in a shorter time frame compared to resource allocation. The dispatcher and on-site supervisor have fixed behaviors, i.e., they know how to proceed. The RL agent experiments actions

with the aim of learning the best possible outcomes. This agent extends the one used in the small case. Therefore, the RL agent will be covered in more detail.

1) THE DISPATCHER

The Dispatcher knows the number of people in need of emergency transportation, at every venue. The ambulance dispatching model uses a storage cell to represent the pool of available ambulances. It outputs all the ambulances at every time step. A distributor right out of the storage allocates the ambulances to each hospital. In case of not needing as many ambulances as available, a third stream sends the remaining ambulances back to availability as if they never left their standby location. Once the ambulances have delivered the patients to the corresponding hospitals, they become immediately available as the traveling time has been already accounted for at the dispatch point. The ambulance allocation is proportional to the number of people waiting at each venue. Thus, if the number of injured people is the same, at both venues, a 50-50 percent split is expected.

During the construction of this model we added a new component to the i2Sim toolbox, an integer distributor. This component was needed for ambulance allocation. It provides a more realistic approach. Without this element the simulation could have fractions of people traveling to different hospitals. With the integer distributor, split ratios are sometimes more indicators than values, e.g., one ambulance available would go to the venue with a ratio higher than 50%.

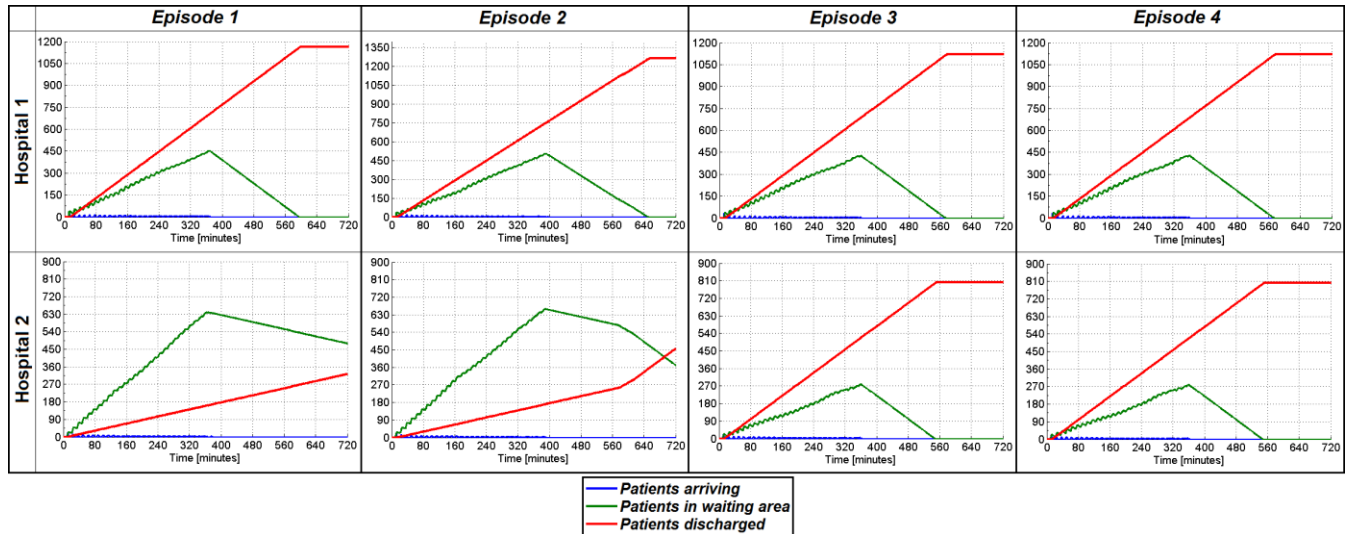


FIGURE 12. Baseline results. Model running with full operational status. Agent's learning progress can be observed in the behavior of the hospitals. Chronological order of the episodes goes from left to right.

2) THE ON-SITE SUPERVISOR

This agent is the simplest of the three and its behavior is rigid i.e., its decisions are situation independent. This on-site supervisor is an agent directing ambulance traffic to a specific hospital from each venue. It is modeled with two fixed ratio distributors. From Venue 1, 60% of the ambulances go to the larger hospital (Hospital 1) and the remaining (40%) to Hospital 2. From Venue 2 ambulances split evenly (50-50) to both hospitals.

3) THE RL AGENT

The RL agent is an expanded version of the formulation described in Section V. By handling more CIs, the state descriptors have multiplied. Likewise, the set of actions is larger. The dimensionality of the problem increased by a factor of 4,169 compared to the proof of concept case. Details about the scaled RL setup are discussed in the next item.

B. THE SEQUENTIAL RL AGENT

Following the same procedure used for the small test case, we approached the agent's representation as an MDP. Many of the features and settings used in the previous case are applicable and remain unchanged. Hence, we emphasize on the changes more than the similarities.

1) STATE SPACE

The state space is formed with the operational indexes (OIs) of the following infrastructures: relevant electrical substations, water station and hospitals. Additionally, we added the availability of electrical backup at each hospital (Yes/No). We selected three out of four substations based on which ones supply the hospitals and the water station. Figure 11 shows the state variables' sources along with their number of states. A combinatorial of the individual OIs yields 8,100,000 states.

The value is significantly higher than the 50,625 possible states in the previous case.

2) ACTION SPACE

The agent manipulates four decision points, three electrical distributors and one for the water supply. Figure 11, identifies those distributors with *DP* and specifies the number of available actions at each point. Like in the test case, a coordinated response considers one action as a combination of settings at all distributors. By combining all decision points, the action space has a total of 396 feasible actions.

3) SETUP AND RESULTS

The size of the LUT, $state\ space \times action\ space$, is 3,207,600,000. As mentioned before, MATLAB uses double as its default datatype. Thus, the size of the LUT is 23.89GB. Due to similarities in formulation with the small test case, the LUT partitions and bands have the same size and repeating patterns listed in Table 2.

Reward generation and policy remain the same. The only difference, in the reward scheme, is the use of a composite signal with the feedback from both hospitals.

We decided to test two different configurations. The first is the baseline, as in the small case, a fully operational environment for which we know the answer. Results for this case are displayed in Figure 12. As seen in the plots, the red line indicates the number of patients treated. This red line has an ascending slope and then flattens, at this point all patients have been treated. The second setup uses an LUT partition 135,000 states apart from the zero and features electrical limitations. The corresponding results are illustrated in Figure 13. Although, this is not yet a distributed version, we had to use LUT strips to exercise different sections of the state space. These tests over LUT partitions enabled us to verify

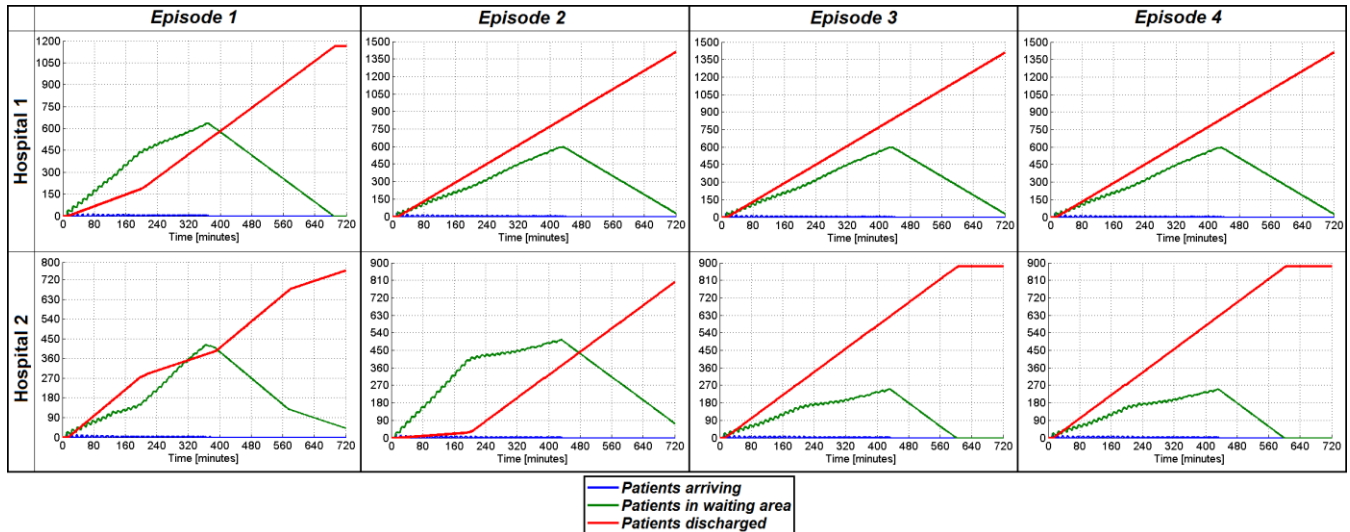


FIGURE 13. Results of baseline model with limited electricity at one substation.

the extra parameters the agent needs for the large distributed test.

This Model was configured to run for 720 timesteps, an equivalent to 12 hours of simulation time. In execution, episodes averaged 85.94 seconds. As seen in the results, we let the training run for four episodes. The plots from episodes 3 and 4 are identical at each hospital. This indicates that the process converged. The agent learned the policy after 3 runs. Thus, we achieved a fast and guaranteed convergence.

C. DISTRIBUTED TRAINING AND THE SCHEDULER

If the totality of the state space was to be explored, we would have over 10,000 partitions to exercise. This translates into 10,000 simultaneous running instances. Even with the available cluster, this partitioning scheme would not offer a good balance between memory and CPU usage. Moreover, the goal of this implementation is to ease the work of decision makers; having results from all possible situations is overwhelming. We decided to run the training using 24 bands. As explained before, a band is a grouping of 4 partitions with sizes: 1125, 900, 675 and 450 respectively. Thus, the number of LUT partitions exercised was 96, in total.

The architecture of the scheduler is the same described in previous sections. One improvement done to the agent was the addition of an offset factor. This enabled the agent to map states from 1 to *maxnumber* (1125, 900, 675 or 450) regardless of the state's value. This parameter was not needed before as the LUT was small. A second addition comprised improvements to parameter handling, specially, LUT partitions and model settings.

As the distributed training happens in the absence of any graphical user interface, running times shrink. Additionally, the iterations (episodes) per partition were reduced to 4. The ϵ -greedy scheme remained unmodified, but when running

4 episodes we used the first 4 exploration values, only (see Table 1). The total training time was 2.86 minutes.

VII. CONCLUSION

The results are satisfactory as they show a very significant increase in performance when compared to all our previous studies. A shaping reward scheme was key in speeding up convergence as it hints the agent towards the global objective. A big part of our efforts, in this study, went on generating the best possible sequential solution. A fair evaluation of a parallel solution only exists when the best sequential version is used.

Moving to the distributed version challenged us in regard to finding the decoupling points. Our choice of exogenous variables to the agent proved to be adequate, as the LUT could be partitioned into fully independent data strips. This enabled us to have a flexible formulation and to choose the desired combination of scenarios for training.

In the beginning, we believed that LUT size was our biggest constraint because of its matrix format. Specially, when simulating large models. But, moving from the aggregate to the large case size did not impact the running time. Our results showed the number of parallel instances as more impactful. A significant demand of CPU resources turns CPU usage into a bottleneck as opposed to memory usage.

In our large case, during the training process, we noticed the small hospital's output to be fluctuating more the large's. This can be attributed to less redundancy in its electrical supply. Overall, this fact did not affect the converging time.

For future work we would like to include multiple RL agents in the same model for testing cooperative roles and a composite goaled optimization. This approach could yield better results in complex cases as it would grow memory-wise while keeping the number of instances unchanged in the

most part. Moreover, this suggestion would make our model optimization more cohesive with the agents communicating one another.

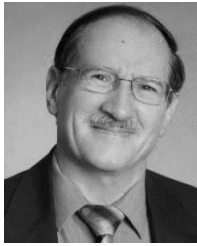
REFERENCES

- [1] E. Bournay. (2009). Trends in natural disasters. Grid Arendal. Accessed: May 10, 2018. [Online]. Available: <http://www.grida.no/resources/7795>
- [2] S. A. Nelson. *Natural Disasters & Assessing Hazards and Risk*. Accessed: May 10, 2018. [Online]. Available: http://www.tulane.edu/~sanelson/Natural_Disasters/introduction.pdf
- [3] T. E. Drabek and G. J. Hoetmer. *Emergency Management: Principles And Practice for Local Government*. Washington, DC, USA: ICMA, 1991.
- [4] P. S. Canada. *An Emergency Management Framework for Canada*, Emergency Manage. Policy Directorate, Ottawa, ON, Canada, 2011.
- [5] Environment Canada, Government of Canada. (Jul. 23, 2013). *Emergency Management Basics*. Accessed: May 11, 2015. [Online]. Available: <https://www.ec.gc.ca/ouragans-hurricanes/default.asp?lang=En&n=31DADD5F-1#archived>
- [6] Federal Emergency Management Agency. (Nov. 21, 2006). *Introduction to Emergency Management*. Accessed: Feb. 23, 2015. [Online]. Available: <https://training.fema.gov/hiedu/docs/fem/chapter>
- [7] *The All-Hazard Plan*, Province Brit. Columbia, Emergency Manage. Brit. Columbia, Victoria, BC, Canada, Nov. 2012.
- [8] *Emergency Management Planning Guide 2010-2011*, Public Safety Canada, Government Canada, Ottawa, ON, Canada, 2010.
- [9] U.S. Department of Homeland Security. (Dec. 8, 2017). *What Is Critical Infrastructure?*. Accessed: May 27, 2018. [Online]. Available: <https://www.dhs.gov/what-critical-infrastructure>
- [10] Public Safety Canada. (Jan. 31, 2018). *National Strategy for Critical Infrastructure*. Accessed: May 3, 2018. [Online]. Available: <https://www.publicsafety.gc.ca/cnt/rsrscs/pblctns/srtg-crtcl-nfrstrctr/index-en.aspx>
- [11] E. Wiseman, T. McLaughlin, and N. Mrad, "Critical infrastructure protection and resilience literature survey: State of the art," Defence Res. Develop. Canada, Ottawa, ON, Canada, Tech. Rep. DRDC Project #CSSP-2013-TI-1039, 2014.
- [12] U. S. Department of Homeland Security. (Jul. 11, 2017). *Critical Infrastructure Sectors*. Accessed: May 27, 2018. [Online]. Available: <https://www.dhs.gov/critical-infrastructure-sectors>
- [13] R. Setola and M. Theodoridou, "Modelling dependencies between critical infrastructures," in *Managing the Complexity of Critical Infrastructures: A Modelling and Simulation Approach*. Cham, Switzerland: Springer, 2016, pp. 19–41.
- [14] S. M. Rinaldi, "Modeling and simulating critical infrastructures and their interdependencies," in *Proc. 37th Annu. Hawaii Int. Conf. Syst. Sci.*, Hawaii, HI, USA, Jan. 2004, p. 8.
- [15] J. R. Martí, "Multisystem simulation: Analysis of critical infrastructures for disaster response," in *The Last Frontier of Complexity (Understanding Complex Systems)*. Cham, Switzerland: Springer, 2014, pp. 255–277.
- [16] M. Ouyang, "Review on modeling and simulation of interdependent critical infrastructure systems," *Rel. Eng. Syst. Safety*, vol. 121, pp. 43–60, Jan. 2014.
- [17] A. De Nicola, G. Vicoli, and M. L. Villani, "A rule-based approach for modelling behaviour in crisis and emergency scenarios," in *Proc. I-ESA Conf.*, London, U.K., vol. 5, 2012, pp. 93–103.
- [18] C. Foglietta, S. Panzieri, and F. Pascucci, "Algorithms and tools for risk/impact evaluation in critical infrastructures," in *Intelligent Monitoring, Control, and Security of Critical Infrastructure Systems*. Berlin, Germany: Springer, 2014, pp. 227–238.
- [19] A. Tofani, G. D'Agostino, and J. Martí, "Phenomenological simulators of critical infrastructures," in *Managing the Complexity of Critical Infrastructures: A Modelling and Simulation Approach*. Cham, Switzerland: Springer, 2016, pp. 85–107.
- [20] J. R. Martí and UBC-JIIRP Group, "The I2Sim simulator for disaster response coordination in interdependent infrastructure systems," *Inst. Comput., Inf., Cogn. Syst.*, Vancouver, BC, Canada, Tech. Rep. jiirp_i2sim_033, 2006.
- [21] *Ministry of Transportation and Infrastructure, British Columbia Smart Infrastructure Monitoring System (BCSIMS)*, Government Brit. Columbia, Victoria, BC, Canada, 2015.
- [22] M. T. Khourj and J. R. Martí, "Modeling critical infrastructure interdependencies in support of the security operations for the Vancouver 2010 Olympics," Defence R&D Canada-CORA, Vancouver, BC, Canada, Tech. Rep. DRDC CORA TR W7714-081111, 2010.
- [23] C. Lopez, "Multi-energy systems simulator for hourly management and optimization of GHG emissions and fuel costs," M.S. thesis, Dept. Elect. Comput. Eng., Univ. Brit. Columbia, Vancouver, BC, Canada, 2011.
- [24] M. Wiering and M. van Otterlo, *Reinforcement Learning: State-of-the-Art*. Berlin, Germany: Springer, 2012.
- [25] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Hoboken, NJ, USA: Wiley, 1995.
- [26] R. S. Sutton, A. G. Barto, and F. Bach, *Reinforcement Learning: An Introduction (Adaptive Computation and Machine Learning)*. Cambridge, MA, USA: MIT Press, 1998.
- [27] P. Kulkarni, *Reinforcement and Systemic Machine Learning for Decision Making*. Hoboken, NJ, USA: Wiley, 2012.
- [28] T. Hester, *TEXPLORE: Temporal Difference Reinforcement Learning for Robots and Time-Constrained Domains*. Cham, Switzerland: Springer, 2013.
- [29] S. Singh, T. Jaakkola, M. L. Littman, and C. Szepesvári, "Convergence results for single-step on-policy reinforcement-learning algorithms," *Mach. Learn.*, vol. 38, no. 3, pp. 287–308, Mar. 2000.
- [30] A. Y. Ng, "Shaping and policy search in reinforcement learning," Ph.D. dissertation, Dept. Comput. Sci., Univ. California, Berkeley, CA, USA, 2003.
- [31] Canadian Association of Emergency Physicians. (2012). *The Canadian Triage and Acuity Scale: Education Manual*. Accessed: Jul. 31, 2018. [Online]. Available: https://caep.ca/wp-content/uploads/2017/06/module_1_slides_v2.5_2012.pdf
- [32] H. J. García, "Multi-hazard risk assessment: An interdependency approach," Ph.D. dissertation, Dept. Civil Eng., Univ. British Columbia, Vancouver, BC, Canada, 2010.
- [33] M. Khourj, C. López, S. Sarkaria, and J. Martí, "Disaster management in real time simulation using machine learning," in *Proc. 24th IEEE Can. Conf. Elect. Comput. Eng. (CCECE)*, Niagara Falls, ON, Canada, May 2011, pp. 001507–001510.
- [34] M. T. Khourj, S. Sarkaria, C. Lopez, and J. Martí, "Reinforcement learning using Monte Carlo policy estimation for disaster mitigation," in *Proc. 8th ICCIP*, Arlington, VA, USA, 2014, pp. 155–172.
- [35] Sumavi. (2010). *xCAT Administrators Guide—Introduction*. Accessed: Jun. 5, 2015. [Online]. Available: <http://sumavi.com/chapters/introduction>
- [36] P. L. Scaringi. (Feb. 14, 2012). *Building A Disaster Resilient Quebec*. Accessed: May 5, 2018. [Online]. Available: https://www.securitepublique.gouv.qc.ca/fileadmin/Documents/securite_civile/colloques/2012/building_disaster_resilient.pdf
- [37] *Action Plan for Critical Infrastructure 2014–2017*, Public Saf. Canada, Government Canada, Ottawa, ON, Canada, 2014.
- [38] T. E. Drabek, "Emergency management and homeland security curricula: Contexts, cultures, and constraints," presented at the Annu. Meeting Western Social Sci. Assoc., Calgary, AB, Canada, Apr. 2007.
- [39] S. A. Nelson. (Aug. 19, 2014). *Natural Disasters & Assessing Hazards and Risk*. Accessed: Aug. 31, 2017. [Online]. Available: http://www.tulane.edu/~sanelson/Natural_Disasters/introduction.pdf
- [40] Federal Emergency Management Agency. (Nov. 21, 2006). *Introduction to Emergency Management*. Accessed: Sep. 23, 2017. [Online]. Available: <https://training.fema.gov/hiedu/docs/fem/chapter>



CESAR LOPEZ (S'09) was born in Bucaramanga, Colombia. He received the bachelor's degree in systems engineering and the Specialization degree in university teaching from the Cooperative University of Colombia in 2001 and 2004, respectively, and the M.A.Sc. degree from The University of British Columbia in 2012, where he is currently pursuing the Ph.D. degree. From 2002 to 2009, he held multiple teaching positions with various universities in Colombia. Since 2017, he has been a

Permanent Instructor with Kwantlen Polytechnic University. He is currently a Licensed Professional Engineer and a member of the Project Management Institute. His research interests include machine learning, critical infrastructure interdependencies, complex systems, simulation, and Internet of Things.



JOSÉ R. MARTÍ (M'79–F'02) was born in Lleida, Spain. He received the Electrical Engineering degree from the Central University of Venezuela in 1971, the MEEPE degree from the Rensselaer Polytechnic Institute, NY, in 1974, and the Ph.D. degree from The University of British Columbia, Canada, in 1981. Since 1988, he has been with The University of British Columbia, where he is currently a Professor of electrical and computer engineering. His special fields of interests include the Electromagnetic Transients Program, real time, parallel processing, system-of-systems simulation, critical infrastructures, and smart grid technologies.



SARBJIT SARKARIA received the Joint Honours degree in applied physics and computer science and the Ph.D. degree in computer science, with a focus on neural networks, from Aston University, Birmingham, U.K., in 1987 and 1991, respectively. He held various positions in the telecommunications and aerospace industries with companies such as MacDonald, Dettwiler & Associates, Nortel, Nokia, and Motorola. Since 1994, he has been holding a sessional post with the Department of Electrical and Computer Engineering, The University of British Columbia, Vancouver, BC, Canada, where he teaches a course on machine learning. He is also a Professional Software Engineer. He is a member of Engineers and Geoscientists BC.

...